

Security Findings Report

Car Rental System

A desktop-based Car Rental Management System developed using Java Swing and MySQL to simplify and organize the car rental process. The application provides a centralized platform for managing vehicles, customer information, rental records and related transactions, reducing manual record keeping and improving the efficiency of rental operations.

The system is designed to streamline everyday car rental activities by providing an intuitive desktop interface and a structured database for storing and managing application data. It helps rental businesses maintain accurate records, track vehicle availability, manage customer details and handle rental operations in a more organized and efficient manner.

Report date:	June 22, 2026
Scope:	White box Testing
Method:	Manual source code review and live exploitation of the running application connected to a MySQL database.
Findings:	5 issues found with 2 Critical, 2 High and 1 Medium level

Summary of Findings:

The Open Worldwide Application Security Project (OWASP) focuses on improving the security of web applications. I performed security testing of Car Rental Management System based on the OWASP Top 10:2025 application security risks.

No	Finding	Severity	CWE
1	Authentication is completely bypassed	CRITICAL	CWE-287
2	SQL Injection in every database query	CRITICAL	CWE-89
3	Hard coded database and admin credentials in source code	HIGH	CWE-798
4	Customer passwords stored in plain text means no hashing	HIGH	CWE-256
5	Broken access control as CNIC used with no ownership check	MEDIUM	CWE-639

Finding 1: Authentication Bypass

CWE: CWE-287 (Improper Authentication)

Location: adminLoginUI.java, custLoginUI.java

Both the admin and customer login screens check the entered credentials, then throw the result away and log the user in unconditionally. The check is performed only for show, the actual if statement that decides whether to log the user in is hard-coded to true.

Vulnerable code:

```
adminLogin.admin(username, password); // return value is never read

if (true) {
    JOptionPane.showMessageDialog(null, "Login successful!");
    new main("admin").setVisible(true); // full admin dashboard is opened
    dispose();
}
```

The identical pattern exists in custLoginUI.java for customer login. In addition, adminLoginUI.java never reads the values typed into the username and password fields at all, it calls the check using two hard-coded instance fields ("admin" / "admin123") that are unrelated to what the user typed so the outcome is the same either way.

Impact: Anyone with access to the application can act as an administrator: add, edit or delete any car or as customer: view any rental history and rent or return cars. This is a complete authentication bypass.

Recommended fix: Replace if (true) with the actual result of the credential check (if

(adminLogin.admin(usernameField.getText(), passwordField.getText()))

and make sure the values passed in are read from the on-screen fields rather than hard coded variable.

Finding 2: SQL Injection

CWE: CWE-89 (SQL Injection)

Location: addCarUI.java, updateCarUI.java, deleteCarUI.java, carHistory.java, carReturn.java, rentPrice.java, rentDialog.java, and others

Every database query in the project is built by directly concatenating raw user input into SQL text instead of using parameterized queries (Prepared Statement). This pattern appears in at least 10 separate locations covering car management (add/update/delete) and all customer facing CNIC lookups (rent, return, rental history, payment).

Vulnerable code:

```
String reg = regField.getText().trim();

String generatedQuery = String.format(
    "DELETE FROM cars WHERE reg_id = '%s'", reg);

DBConnect.executeStatementUpdate(generatedQuery);
```

```
String sql = "SELECT r.user_cnic, c.*, r.rent_date FROM cars c " +
    "JOIN rented_cars r ON c.reg_id = r.reg_id " +
    "WHERE r.user_cnic = '" + cnic + "'";
```

```
Attacker types into 'Enter your CNIC': x' OR '1'='1
Resulting SQL:
SELECT r.user_cnic, c.* FROM cars c JOIN rented_cars r
ON c.reg_id=r.reg_id WHERE r.user_cnic = 'x' OR '1'='1'
-> leaked row for real CNIC: 35202-99999999-9
```

Impact: disclosure of other customers personal rental records (CNIC national ID number, cars rented, dates). The same injection pattern in rentDialog.java, carReturn.java and rentPrice.java can similarly be used to manipulate or read rental and payment records belonging to other customers and the same pattern in the admin car management screens can be used to modify or delete arbitrary rows or potentially read or modify data outside the car rental database entirely.

Recommended fix. Replace every Statement built with string concatenation with a Prepared Statement for all user supplied values, in every file listed above.

Finding 3: Hard-Coded Credentials

CWE: CWE-798 (Use of Hard-coded Credentials)

Location: DBConnect.java, /secondDB.java, adminLogin.java, adminLoginUI.java

Database credentials and the admin password are written directly into the source code as string literals rather than being read from a configuration file, environment variable or secrets manager.

DBConnect.java:

```
String URL = "jdbc:mysql://localhost:3306/car_rental";
String USER = "root";
String PASS = "root";
```

adminLogin.java and adminLoginUI.java:

```
if( username.equals("admin") && password.equals("admin123")){ ... }

// adminLoginUI.java
String username = "admin";
String password = "admin123";
```

Impact. Anyone who can view the source code obtains full database credentials and the admin password immediately. If this database account is ever reused or the same MySQL root pattern is reused on a real server this is a direct path to full database compromise.

Recommended fix. Read database credentials from an environment variable. Store the admin password as a salted hash instead of a literal string.

Finding 4: Unprotected Storage of Credentials

CWE: CWE-256 (Unprotected Storage of Credentials) / CWE-522

Location: regCustomer.java, custLogin.java, custExist.java, data file abc.txt

Customer accounts are stored as plain comma separated lines in a text file on disk with the password saved and compared as plain text, no hashing, salting or encryption of any kind.

regCustomer.java:

```
BufferedWriter bw = new BufferedWriter(new FileWriter(CUSTOMER_FILE, true));
bw.write(username + "," + email + "," + password); // password written in clear text
```

(abc.txt):

```
abd, abc@gmail.com, 123
salman, salman@gmail.com, 1234
```

Impact. Anyone with read access to this file obtains every customer's password in plain text. Because people frequently reuse passwords, this also puts customers accounts on other, unrelated services at risk.

Recommended fix. Never store passwords in plain text. Hash each password with a slow, salted algorithm designed for credentials (e.g. bcrypt, scrypt or Argon2) before saving it, and compare hashes on login rather than comparing plain text.

Finding 5: Broken Access Control

CWE: CWE-639 (Authorization Bypass Through User-Controlled Key)

Location: useCases.java, rentDialog.java, carReturnUI.java, carHistory.java

Once logged in as any customer, every rental action (renting a car, returning a car, viewing rental history, viewing payment amounts) is performed by simply typing in a CNIC (national ID number) with no verification that it belongs to the logged-in user. The CNIC is treated as if it were a secret but it is only ever asked for in a plain text prompt and never checked against the logged in account.

useCases.java:

```
String cnic = JOptionPane.showInputDialog(useCases.this, "Enter your CNIC:");
if (cnic != null && !cnic.trim().isEmpty()) {
    JFrame historyFrame = new carHistoryUI(cnic.trim()); // no ownership check at all
    historyFrame.setVisible(true);
}
```

Impact. Any logged in customer can view any other customer's complete rental history or return / rent a car under someone else's CNIC, simply by typing a different, guessable format of ID.

Recommended fix. Tie rental actions to the currently authenticated account not to a value retyped on every screen. If CNIC must be reentered for verification purposes, cross-check it against the CNIC on file for the logged in account before proceeding.